BLOQUE III - TEMA 5

DESARROLLO DE SISTEMAS

Arquitectura Java EE/Jakarta EE y plataforma.NET: componentes, persistencia y seguridad. Características, elementos, lenguajes y funciones en ambos entornos. Desarrollo de Interfaces.

Supera. by thePower

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2024.

ÍNDICE

1. INTRODUCCIÓN GENERAL A LAS PLATAFORMAS EMPRESARIALES 5
2. ARQUITECTURA MULTICAPA 7
3. ARQUITECTURA DE JAVA EE/JAKARTA EE 8
3.1 Tecnologías de la plataforma Jakarta EE 8
3.2 Plataformas de desarrollo Java 9
3.3. Compilación 10
3.4. Contenedores 11
3.5. Servlets 13
3.6. Enterprise JavaBeans (EJB) 13
3.7. Herramientas 14
3.7.1. Servidores de aplicaciones 14
3.7.2. Entornos de desarrollo (IDEs) 15
3.7.3. Herramientas de construcción y automatización 15
3.7.4. Frameworks 16
3.8. Persistencia Jakarta EE 16
3.8.1. JDBC (Java Database Connectivity) 16
3.8.2. JPA (Java Persistence API) 17
3.9. Gestión de transacciones y conexión a bases de datos 17
3.9.1. Gestión de transacciones. 17
3.9.2. Conexión a bases de datos 17
3.10. Seguridad. JAAS 18
4. ARQUITECTURA DE LA PLATAFORMA .NET 19
4.1 Componentes principales 19
4.2 Capas y modelos de aplicación 20
4.3. Persistencia de datos en .NET: ADO.NET y Entity Framework 20
4.3.1. ADO.NET 20
4.3.2. Entity Framework (EF) 21

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

4.4. Seguridad en .NET 22
4.5. Análisis del modelo de acceso a datos ADO.NET 22
4.5.1. Biblioteca de Clases Base (BCL) 22
4.5.2. Common Language Runtime (CLR) 22
4.5.3. Lenguajes y compilación al formato intermedio común (CIL) 23
4.5.4. ADO.NET 24
4.6. Gestión de paquetes y tipos de datos en .NET 26
5. LENGUAJES, TECNOLOGÍAS Y HERRAMIENTAS 27
5.1 Lenguajes 27
5.2 Herramientas de desarrollo 28
6. DESARROLLO DE INTERFACES DE USUARIO 29
6.1 Java EE/Jakarta EE 29
6.2 .NET 29

Significado de los iconos que aparecen dentro de los TEMAS:



Examen


Importante


Recordatorio


Atención

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025.

1. INTRODUCCIÓN GENERAL A LAS PLATAFORMAS EMPRESARIALES

Las plataformas empresariales proporcionan un conjunto de tecnologías, herramientas y estándares para el desarrollo, despliegue y ejecución de aplicaciones empresariales. Estas aplicaciones suelen tener una gran complejidad, necesidad de escalabilidad, alta disponibilidad y seguridad. Las plataformas Java EE (hoy **Jakarta EE**) y **.NET** son dos de las soluciones más extendidas para el desarrollo de software empresarial.

Una **plataforma empresarial** de desarrollo de software es un conjunto estructurado de tecnologías, herramientas, lenguajes y componentes que permiten construir, desplegar y ejecutar aplicaciones robustas, escalables y mantenibles, generalmente orientadas a entornos corporativos o institucionales.

- Están diseñadas para soportar **altos volúmenes de usuarios y transacciones**.
- Permiten la separación en **capas** (presentación, lógica de negocio, persistencia).
- Integran mecanismos de **seguridad, transaccionalidad, concurrencia y escalabilidad**.
- Facilitan la **interoperabilidad** mediante **servicios web** y tecnologías estándar.

Las dos plataformas más extendidas en el ámbito empresarial son:

- **Java EE (ahora Jakarta EE)**, impulsada originalmente por Sun Microsystems y actualmente por la Eclipse Foundation.
- **.NET**, desarrollada por Microsoft y orientada al ecosistema Windows, aunque desde .NET Core es completamente multiplataforma.

Java EE / Jakarta EE

- **Java EE** (Java Platform, Enterprise Edition), renombrado como **Jakarta EE** desde 2019 tras pasar de Oracle a la Eclipse Foundation, es una plataforma basada en el lenguaje **Java** para el desarrollo de aplicaciones distribuidas y escalables.
- Está orientada a entornos empresariales complejos, facilitando el desarrollo de aplicaciones **multicapa** mediante componentes como **Servlets, JSP, EJB o JPA.**
    - **Servlets y JSP** para la capa web.
    - **EJB (Enterprise JavaBeans)** para la lógica de negocio.
    - **JPA (Java Persistence API)** para el acceso a datos.
- Proporciona un conjunto estandarizado de **APIs** y servicios para gestionar seguridad, persistencia, transacciones, servicios web, mensajería, etc.
- Su objetivo es separar las distintas capas de la aplicación (presentación, lógica de negocio, datos) y favorecer la **modularidad, portabilidad y reutilización** del código.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

.NET

- **.NET** es una plataforma de desarrollo creada por **Microsoft**, inicialmente para sistemas Windows, aunque desde la aparición de **.NET Core** (y ahora **.NET 5+** como evolución unificada) es multiplataforma (Windows, Linux, macOS).
- Permite crear todo tipo de aplicaciones (web, escritorio, móviles, servicios web, microservicios) mediante lenguajes como **C#**, **VB.NET** o **F#**.
- Ofrece un entorno común de ejecución, el **Common Language Runtime (CLR)**, y un amplio conjunto de bibliotecas base que simplifican la programación empresarial.
- Su finalidad es ofrecer una plataforma robusta y productiva para el desarrollo **rápido, seguro y escalable** de software empresarial.

Característica	Java EE / Jakarta EE	.NET (Framework / Core / 5+)
Origen	Oracle / Eclipse Foundation	Microsoft
Lenguaje principal	Java	C#, VB.NET, F#
Multiplataforma	Sí (por definición)	Sí (.NET Core y posteriores)
Entorno de ejecución	Máquina Virtual Java (JVM)	Common Language Runtime (CLR)
Modelo de componentes	EJB, Servlets, JSP, JPA	ASP.NET, WCF, Entity Framework
Gestión de dependencias	Maven, Gradle	NuGet
IDE más común	Eclipse, IntelliJ IDEA	Visual Studio
Licencia	Open Source (Eclipse Public License)	Open Source (MIT License)
Despliegue	Servidores de aplicaciones (WildFly, etc.)	Self-hosted o servidores IIS, Kestrel
Ecosistema	Muy amplio, con frameworks como Spring	Muy integrado con Azure, Blazor, MAUI
Seguridad	JAAS, roles, filtros	ASP.NET Identity, Windows Auth

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

2. ARQUITECTURA MULTICAPA

La **arquitectura multicapa** es un modelo de diseño de software que organiza una aplicación en **capas separadas**, donde cada capa tiene una **responsabilidad específica**. Este enfoque favorece la **modularidad, el mantenimiento, la escalabilidad y la reutilización** del código.

En el desarrollo empresarial con plataformas como **Jakarta EE** o **.NET**, la arquitectura multicapa es el modelo más común y se estructura habitualmente en tres o cuatro capas.

Capas típicas en una arquitectura multicapa

1. **Capa de presentación (cliente o interfaz de usuario)**
   - Es la capa con la que interactúa el usuario final.
   - Se encarga de mostrar datos y recoger entradas del usuario.
2. **Capa de lógica de negocio (o de servicios)**
   - Contiene las **reglas del negocio**, validaciones, cálculos y flujos de trabajo.
   - En Jakarta EE se implementa mediante **EJBs** o clases de servicios con CDI.
   - En .NET se usa con clases de servicios o controladores en ASP.NET.
3. **Capa de acceso a datos (persistencia)**
   - Gestiona el acceso a las bases de datos, el almacenamiento y la recuperación de datos.
   - Jakarta EE usa **JPA** o **JDBC**, mientras que .NET usa **Entity Framework** o **ADO.NET**.
   - Aplica el patrón DAO (Data Access Object) para separar lógica de acceso a datos del resto de la aplicación.
2. **Capa de base de datos (o de recursos)**
   - Contiene los datos persistentes. Puede ser una base de datos relacional (PostgreSQL, SQL Server, Oracle) u otros recursos como colas, ficheros o APIs externas.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

3. ARQUITECTURA DE JAVA EE/ JAKARTA EE

Java EE, actualmente conocido como **Jakarta EE**, es una plataforma estándar para el desarrollo de aplicaciones empresariales basadas en el lenguaje Java. Su arquitectura se basa en la separación de responsabilidades en **capas bien definidas**, el uso de **componentes reutilizables** y la ejecución en contenedores.

3.1 TECNOLOGÍAS DE LA PLATAFORMA JAKARTA EE

La plataforma **Jakarta EE** está compuesta por un conjunto de especificaciones y APIs que proporcionan funcionalidades esenciales para el desarrollo de **aplicaciones empresariales modulares, distribuidas y escalables**.

Tecnologías principales de Jakarta EE:

- **Servlet API:** Permite gestionar **peticiones HTTP** desde el navegador del cliente. Es la base de las aplicaciones web tradicionales. **Se ejecutan en el contenedor web del servidor de aplicaciones**.
- **JSP (JavaServer Pages):** Tecnología de presentación que permite incrustar código Java en páginas HTML. Aunque ha caído en desuso frente a JSF y frameworks modernos, aún se encuentra en aplicaciones heredadas.
- **JSF (JavaServer Faces):** Framework orientado a componentes para construir interfaces web interactivas. Incluye soporte para validación de formularios, navegación y binding con beans de respaldo.
- **CDI (Contexts and Dependency Injection):** Estándar para inyección de dependencias y gestión del ciclo de vida de los componentes. Promueve la escritura de código desacoplado y más mantenible.
- **EJB (Enterprise JavaBeans):** Tecnología para encapsular la lógica de negocio. Permite declarar componentes que gestionan automáticamente transacciones, seguridad, concurrencia, etc.
- **Bean Validation:** Permite declarar restricciones de validación directamente en las clases Java mediante anotaciones (@NotNull, @Size, @Email...).

Principales APIs en Java EE / Jakarta EE



El término **API (Application Programming Interface)** se refiere al conjunto de **interfaces, clases y anotaciones** que forman parte del estándar y que los desarrolladores utilizan para construir aplicaciones empresariales.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

API	Funcionalidad principal
Servlet API	Gestión de peticiones HTTP en la capa web.
JSP (JavaServer Pages)	Generación de HTML dinámico embebiendo código Java.
JSF (JavaServer Faces)	Framework de componentes UI para aplicaciones web.
CDI (Contexts and Dependency Injection)	Inyección de dependencias y control de ciclo de vida.
EJB (Enterprise JavaBeans)	Lógica de negocio distribuida, transacciones, concurrencia, seguridad.
JPA (Java Persistence API)	API para el acceso y persistencia de datos mediante el modelo ORM (Object-Relational Mapping). Permite mapear clases Java a tablas de bases de datos relacionales.
JAX-RS	Creación de servicios web RESTful. API que proporciona soporte a la creación de servicios web REST.
JAX-WS	Servicios web basados en SOAP (más verbosos y estructurados).
JMS (Jakarta Messaging)	API para comunicación asíncrona entre componentes mediante colas y temas (publish-subscribe), muy útil en entornos desacoplados.
Jakarta Batch	API para el procesamiento por lotes, utilizada para ejecutar tareas programadas o de larga duración, como importaciones masivas de datos.
Jakarta Bean Validation	Validación declarativa de datos mediante anotaciones (@NotNull, @Email, etc.).
Jakarta Security	Gestión de autenticación, autorización y políticas de seguridad en aplicaciones.
Jakarta Transactions (JTA)	API para gestionar transacciones distribuidas o locales.
Jakarta Mail	Envío y recepción de correo electrónico desde aplicaciones Java.
Jakarta JSON Processing / Binding	Manejo de datos en formato JSON: lectura, escritura y mapeo a objetos Java.
StAX (Streaming API for XML)	Es una API de Java para el procesamiento eficiente y en tiempo real de documentos XML. Permite leer y escribir XML utilizando un modelo de acceso por eventos.
SAX (Simple API for XML) (*)	Es una API de análisis secuencial y basada en eventos
JDOM (*)	Es una API orientada a objetos para trabajar con XML en Java.
JAAS	API de JAVA proporciona un sistema de autenticación y control de acceso de usuarios y grupos para una aplicación:
JAF	JavaBeans Activation Framework, para manejar tipos de contenido MIME.
JACC	Java Authorization Contract for Containers, define políticas de autorización para contenedores Java EE, pero no se usa directamente en aplicaciones.
JNDI	Java Naming and Directory Interface, permite acceder a servicios de nombres (como LDAP), pero no gestiona autenticación en sí misma.

Anotaciones para distinguir JAX-WS (SOAP) y JAX-RS (REST):

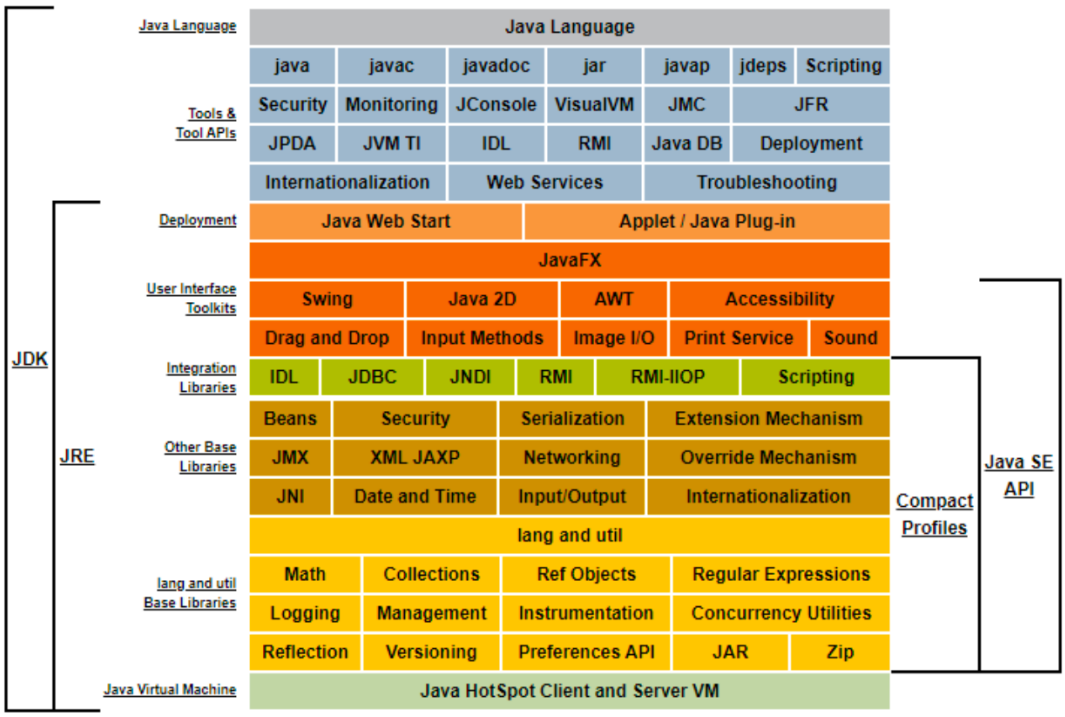
Tecnología	Anotaciones básicas	Función
JAX-WS (SOAP)	@WebService, @WebMethod, @WebParam	Exponen una clase como servicio web SOAP, sus métodos como operaciones y sus parámetros.
JAX-RS (REST)	@Path; métodos HTTP: @GET, @POST, @PUT, @DELETE; @Produces / @Consumes	Definen la ruta del recurso REST, el método HTTP que atiende y el formato de los datos que produce o consume.

3.2 PLATAFORMAS DE DESARROLLO JAVA

El término "Java" no hace referencia únicamente al lenguaje de programación, sino también a todo un **ecosistema de plataformas** que permiten construir aplicaciones para distintos tipos de dispositivos y entornos.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- **Java SE (Standard Edition)**: Proporciona el conjunto básico de bibliotecas y APIs necesarias para desarrollar aplicaciones generales en Java. Es la base sobre la que se construyen las demás ediciones.
- **Java EE (Enterprise Edition)**, actualmente conocida como **Jakarta EE**, amplía las funcionalidades de Java SE incorporando APIs específicas para el desarrollo de aplicaciones empresariales complejas, como servidores web, servicios distribuidos o gestión de transacciones.
- **Java ME (Micro Edition)**: Diseñada para entornos con recursos limitados, como teléfonos móviles, dispositivos embebidos o electrodomésticos inteligentes. Es una versión reducida de Java SE adaptada a estas plataformas.
- **JavaFX**: Es un conjunto de herramientas orientado a crear interfaces gráficas enriquecidas (RIA, Rich Internet Applications). Ofrece un modelo moderno para el diseño de interfaces de usuario interactivas, utilizando una API ligera y versátil.



3.3. COMPILACIÓN

El proceso de compilación en **Jakarta EE** implica varios pasos que convierten el código fuente Java en aplicaciones ejecutables sobre un servidor de aplicaciones. Se compila directamente a código máquina, en Java/Jakarta EE se realiza una compilación a un formato intermedio conocido como **bytecode**, que se ejecuta en la **Java Virtual Machine (JVM)**.

Fases del proceso de compilación

1. **Compilación del código fuente:** Se crean los archivos compilados que **contienen bytecode,** un código intermedio independiente de la plataforma.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

2. **Empaquetado del proyecto**: Este empaquetado puede hacerse con herramientas como **Maven, Gradle o Ant.**
    - .jar (Java Archive) para bibliotecas.
    - .war (Web Application Archive) para aplicaciones web.
    - .ear (Enterprise Archive) para aplicaciones empresariales que incluyen módulos web y EJB.

2. **Despliegue en el servidor de aplicaciones**
    - El archivo .war o .ear se despliega en un **servidor compatible con Jakarta EE** (GlassFish, Payara o WildFly).
    - El servidor contiene un **contenedor** que interpreta el bytecode y gestiona el ciclo de vida de los componentes (Servlets, EJB, etc.).

4. **Ejecución Just-In-Time (JIT)**



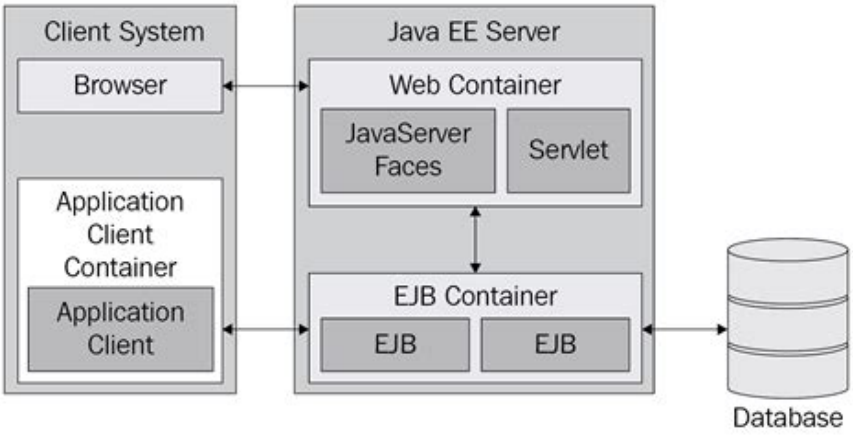
- La JVM no ejecuta directamente el bytecode, sino que lo convierte a código máquina **en tiempo de ejecución** mediante un **compilador JIT** (Just-In-Time).

3.4. CONTENEDORES

La ejecución de aplicaciones Jakarta EE se realiza sobre un **servidor de aplicaciones**, que proporciona los **contenedores** necesarios para **gestionar los distintos tipos de componentes:**

- **Contenedor web**: Ejecuta Servlets, JSP, JSF y gestiona las peticiones HTTP.
- **Contenedor EJB**: Ejecuta los componentes EJB, proporcionando servicios como transacciones, seguridad, concurrencia, etc.
- **Contenedor de persistencia**: Gestiona entidades JPA y las conexiones a la base de datos.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025



En la plataforma **Jakarta EE**, los **contenedores** son componentes del **servidor de aplicaciones** encargados de proporcionar el entorno de ejecución a los diferentes tipos de componentes de una aplicación (servlets, EJB, etc.). Cada contenedor gestiona automáticamente aspectos como el ciclo de vida, la seguridad, la inyección de dependencias y la concurrencia, liberando al desarrollador de tareas repetitivas y complejas.

- **Contenedor web:** El contenedor web es el responsable de ejecutar los componentes web, como:
  - **Servlets**
  - **JSP (JavaServer Pages)**
  - **JSF (JavaServer Faces)**
- **Contenedor EJB:** El contenedor EJB es el entorno de ejecución para los componentes de negocio llamados Enterprise JavaBeans. Es parte esencial del servidor de aplicaciones Jakarta EE completo (por ejemplo, WildFly, Payara, GlassFish).
  - **Tipos de beans gestionados:**
    - **Session Beans (stateless, stateful, singleton)**
    - **Message-Driven Beans (MDB)**



Diferencia clave: mientras que el contenedor web se centra en la interfaz de usuario y el protocolo HTTP, el contenedor EJB se orienta a la lógica empresarial, servicios internos, transacciones y comunicación entre componentes.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

3.5. SERVLETS

Un **Servlet** es un componente Java que se ejecuta en el **servidor** y permite generar contenido dinámico, generalmente en respuesta a peticiones **HTTP**. Forma parte de la especificación **Jakarta Servlet** y constituye la base sobre la que se construyen muchas aplicaciones web en Jakarta EE.

Los Servlets se ejecutan dentro de un **contenedor web** (Tomcat, Jetty, GlassFish), que se encarga de gestionar su ciclo de vida, recibir peticiones, invocar al servlet y enviar la respuesta al cliente.

Funciones principales de un Servlet

- Procesar solicitudes HTTP (GET, POST, PUT, DELETE...).
- Generar contenido dinámico (HTML, XML, JSON, etc.).
- Gestionar sesiones de usuario.
- Servir como controlador en arquitecturas tipo MVC.
- Enviar redirecciones o reenviar peticiones a otros recursos (páginas JSP, otros Servlets, etc.).

Configuración y mapeo de un Servlet

- **Descriptor de despliegue (web.xml):** archivo XML de configuración de la aplicación web (**Deployment Descriptor**), en el que se declaran los Servlets y se mapean a las URL que atienden.
- **Anotación @WebServlet:** introducida en **Servlet 3.0**, permite mapear un Servlet directamente en el código Java, sin necesidad de XML. Ejemplo: @WebServlet("/saludo").

Ciclo de vida de un Servlet

El ciclo de vida de un Servlet está gestionado por el contenedor web y consta de las siguientes fases:

1. **Carga**: el contenedor carga la clase del servlet (puede ser al iniciar la aplicación o tras la primera petición).
2. **Instanciación**: se crea una instancia de la clase Servlet.
3. **Inicialización (init())**: se ejecuta una única vez para configurar el servlet.
4. **Atención a peticiones (service())**: se llama cada vez que llega una petición. Según el método HTTP, se invoca doGet(), doPost(), etc.
5. **Destrucción (destroy())**: se ejecuta una única vez antes de eliminar el servlet de memoria.

3.6. ENTERPRISE JAVABEANS (EJB)

Enterprise JavaBeans (EJB) es una especificación que define un modelo estándar para desarrollar **componentes de negocio en el lado del servidor**. Estos componentes, conocidos como ***enterprise beans,*** encapsulan la lógica empresarial y se ejecutan dentro de un **contenedor EJB**, que proporciona servicios como gestión de transacciones, seguridad, concurrencia y más.

Tipos de EJB

Existen varios tipos de *EJB*, cada uno diseñado para casos de uso específicos:

1. **Session Beans**: Gestionan tareas relacionadas con la lógica de negocio y pueden ser:
   - **Stateless (sin estado)**: No mantienen información entre invocaciones. Son ideales para operaciones independientes y de corta duración.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- **Stateful (con estado)**: Mantienen información de estado entre invocaciones del mismo cliente. Útiles para procesos que requieren mantener datos entre pasos, como un carrito de compras.
- **Singleton**: Existe una única instancia compartida en toda la aplicación. Se utilizan para tareas que requieren un estado compartido global, como la gestión de configuraciones.

2. **Message-Driven Beans (MDB)**: Procesan mensajes de forma asíncrona, generalmente provenientes de sistemas de mensajería como Jakarta Messaging (JMS).



Las clases e interfaces de **Enterprise JavaBeans (EJB)** se encuentran tradicionalmente en el **paquete javax.ejb (Java EE).** En **Jakarta EE**, tras el cambio de gobernanza a la Eclipse Foundation, este paquete ha pasado a denominarse **jakarta.ejb.**

3.7. HERRAMIENTAS

3.7.1. Servidores de aplicaciones

Los **servidores de aplicaciones Jakarta EE** implementan las especificaciones del estándar y proporcionan contenedores web y EJB:

- **GlassFish**: Servidor oficial de referencia para Jakarta EE. Ofrece soporte completo de las especificaciones, desarrollo activo y buena integración con Eclipse.
- **Payara Server**: Derivado de GlassFish, con mejoras de rendimiento y soporte profesional. **Orientado a entornos de producción.**
- **WildFly (antes JBoss)**: es un servidor de aplicaciones ligero y flexible, mantenido por Red Hat. Muy utilizado en entornos empresariales.
  - Ligero, modular y adecuado para aplicaciones empresariales complejas.
- **Apache TomEE**:
  - Basado en Tomcat, pero ampliado con soporte completo para Jakarta EE.
  - Útil en proyectos que quieren aprovechar la simplicidad de Tomcat con más funcionalidades empresariales.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025



- **WebLogic (Oracle) y WebSphere (IBM)**:
    - Soluciones comerciales con alto rendimiento y escalabilidad.

3.7.2. Entornos de desarrollo (IDEs)

Los IDEs facilitan la escritura de código, la depuración, el control de versiones y el despliegue de aplicaciones:

- **Eclipse IDE for Enterprise Java Developers**:
    - Gratuito y de código abierto.
    - Soporte para Maven, Git, servidores Jakarta EE, y herramientas gráficas de desarrollo.
- **IntelliJ IDEA**:
    - Versión **Community** gratuita y **Ultimate** de pago.
    - Muy valorado por su productividad y soporte para tecnologías como Spring, JPA, JSF y más.
- **Apache NetBeans**:
    - IDE oficial del proyecto Apache.
    - Buena integración con GlassFish y Jakarta EE, aunque menos popular que Eclipse o IntelliJ.

3.7.3. Herramientas de construcción y automatización

Estas herramientas gestionan la compilación, dependencias, empaquetado y despliegue:

- **Apache Maven**:
    - Herramienta estándar para la construcción de proyectos Java.
    - Utiliza un archivo pom.xml para definir dependencias, plugins y fases de construcción.
- **Gradle**:
    - Alternativa moderna a Maven, basada en un lenguaje de scripts (Groovy/Kotlin).
    - Más flexible, ideal para proyectos grandes o con configuraciones personalizadas.
    - A diferencia de Maven, que usa **pom.xml**, Gradle configura las dependencias y tareas de construcción mediante el archivo **build.gradle** (escrito en lenguajes como Groovy o Kotlin DSL; en este último caso, el archivo se denomina build.gradle.kts).
- **Apache Ant**:
    - Predecesora de Maven.
    - Menos usada hoy día, pero aún presente en algunos entornos.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

3.7.4. Frameworks

Aunque Jakarta EE proporciona un conjunto estandarizado de APIs para desarrollar aplicaciones empresariales en Java, en la práctica, muchos desarrolladores y organizaciones emplean frameworks complementarios que simplifican y potencian el desarrollo. Entre ellos destaca el ecosistema Spring.

- **Spring Framework**: **framework de código abierto** para el desarrollo de aplicaciones Java. Fue diseñado como una alternativa ligera a EJB (Enterprise JavaBeans) y se ha convertido en un estándar para el desarrollo Java empresarial moderno.
    - Alternativa ligera a EJB, con soporte para inyección de dependencias, transacciones, seguridad, REST, etc.
    - **Programación orientada a aspectos (AOP).**
    - Acceso simplificado a bases de datos (JDBC, ORM).
    - Módulos independientes: Spring Core, Spring MVC, Spring Security, Spring Data, etc.



- **Spring Boot:** Spring Boot es una extensión del Spring Framework que permite crear aplicaciones Java autocontenidas, es decir, que incluyen su propio servidor embebido (como Tomcat o Jetty), facilitando el despliegue sin necesidad de empaquetar en WAR y desplegar en un servidor externo.

- **Hibernate**: Framework de ORM que inspiró la especificación JPA.
- **PrimeFaces / OmniFaces / RichFaces**:
    - Bibliotecas de componentes UI para JSF.
    - Permiten construir interfaces web modernas y con interacción avanzada.
- **Arquillian**:
    - Herramienta para pruebas de integración en Jakarta EE.
    - Permite ejecutar pruebas en el propio contenedor o servidor.

3.8. PERSISTENCIA JAKARTA EE

La **persistencia de datos** es un aspecto **esencial** en el desarrollo de aplicaciones empresariales. Consiste en **almacenar de forma permanente la información** utilizada por las aplicaciones, generalmente en bases de datos relacionales. En el contexto de Jakarta EE, la persistencia de datos se gestiona principalmente mediante **JPA (Java Persistence API)** y, en menor medida, con **JDBC**.

3.8.1. JDBC (Java Database Connectivity)

JDBC es la **API básica de Java** para el **acceso a bases de datos relacionales** mediante sentencias **SQL**. Permite:

- Conectarse a una base de datos mediante un driver JDBC.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- Ejecutar consultas (SELECT) y actualizaciones (INSERT, UPDATE, DELETE).
- Recuperar los resultados y procesarlos con objetos como ResultSet.

3.8.2. JPA (Java Persistence API)



JPA es la especificación estándar de Jakarta EE para la **persistencia de objetos Java en bases de datos relacionales** usando el patrón **ORM (Object-Relational Mapping).**

Características de JPA:

- Representa tablas como **clases Java** y filas como **instancias (objetos)**.
- Usa **anotaciones** como @Entity, @Id, @OneToMany, etc.
- Permite realizar consultas con **JPQL** (Java Persistence Query Language), un lenguaje orientado a objetos.

El archivo estándar de configuración de JPA es **persistence.xml**, ubicado en el directorio **META-INF**. En él se define la **Persistence Unit** (Unidad de Persistencia), que agrupa las clases de entidad (@Entity) y los datos de conexión al SGBD.

3.9. GESTIÓN DE TRANSACCIONES Y CONEXIÓN A BASES DE DATOS

3.9.1. Gestión de transacciones.

Una **transacción** es una unidad de trabajo que debe ejecutarse de forma **atómica**, **consistente**, **aislada** y **duradera** (propiedades ACID). Si una parte de la operación falla, toda la transacción debe deshacerse para mantener la coherencia del sistema. En Jakarta EE:

- Usa la especificación **JTA (Jakarta Transactions API)**.
- Puede ser **declarativa** o **programática**.

3.9.2. Conexión a bases de datos

La **conexión a bases de datos** debe ser eficiente, segura y reutilizable. Ambos entornos emplean **pools de conexiones** para mejorar el rendimiento evitando aperturas y cierres repetidos. En Jakarta EE:

- Las conexiones se configuran mediante un **DataSource JNDI**, gestionado por el servidor de aplicaciones.
- Se usa la anotación @PersistenceContext para inyectar un EntityManager con una conexión gestionada.
- El servidor gestiona automáticamente el pool de conexiones.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

3.10. SEGURIDAD. JAAS

JAAS (Java Authentication and Authorization Service)

- Es la API estándar de Java para la autenticación y autorización de usuarios.
- Permite establecer **módulos de autenticación** que validan credenciales (usuario/contraseña, certificados, etc.).
- Soporta distintos backends de identidad: LDAP, bases de datos, servicios externos.
- Los permisos se pueden definir mediante **políticas declarativas** (.policy) o a nivel de código.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

4. ARQUITECTURA DE LA PLATAFORMA .NET

La plataforma **.NET** es un entorno de desarrollo creado por Microsoft que permite construir aplicaciones de todo tipo (web, escritorio, móviles, servicios y microservicios). Desde la aparición de **.NET Core** y su evolución a **.NET 5 y versiones posteriores**, se ha convertido en una plataforma de **código abierto, multiplataforma y modular**.

4.1 COMPONENTES PRINCIPALES

- **.NET Core / .NET (5, 6, 7, 8...)**: Nueva generación del framework .NET, **multiplataforma** (Windows, Linux, macOS), **modular**, **de código abierto** y unificado. Sustituye al .NET Framework tradicional (limitado a Windows).
- **ASP.NET / ASP.NET Core**: Framework para el desarrollo de aplicaciones web y APIs REST. ASP.NET Core es una versión ligera y de alto rendimiento, ideal para microservicios y aplicaciones modernas.
- **Entity Framework**: ORM (Object-Relational Mapping) para trabajar con bases de datos relacionales usando objetos .NET. Permite generar modelos de datos y realizar consultas sin escribir SQL.
- **Windows Communication Foundation (WCF)**: Tecnología orientada a servicios (SOA) para la comunicación entre aplicaciones distribuidas, aunque está siendo sustituida en muchos casos por servicios REST.
- **Windows Presentation Foundation (WPF)** y **Windows Forms**: Para aplicaciones de escritorio en Windows. WPF permite interfaces modernas y con separación clara entre lógica y presentación (MVVM).
- **Blazor**: Framework para construir interfaces web interactivas con C# en lugar de JavaScript, ejecutándose en el navegador gracias a WebAssembly o en el servidor.
- **MAUI (.NET Multi-platform App UI)**: Evolución de Xamarin.Forms, permite crear aplicaciones móviles y de escritorio desde una única base de código.

Ciclo de vida de las versiones de .NET

A partir de **.NET 5**, Microsoft lanza una versión principal cada año, en **noviembre**. Las versiones **pares** (.NET 6, .NET 8, .NET 10) tienen soporte a largo plazo (**LTS**, 3 años), mientras que las **impares** (.NET 5, .NET 7, .NET 9) tienen soporte estándar (**STS**). El soporte STS era de **18 meses**, pero en septiembre

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

de 2025 Microsoft lo amplió a **24 meses** a partir de .NET 9.

Versión	Tipo de soporte	Duración del soporte
.NET 5	STS	18 meses
.NET 6	LTS	3 años
.NET 7	STS	18 meses
.NET 8	LTS	3 años
.NET 9	STS	24 meses (primera versión con la ampliación)
.NET 10	LTS	3 años

4.2 CAPAS Y MODELOS DE APLICACIÓN

En .NET, la arquitectura típica de una aplicación empresarial se organiza en capas para separar responsabilidades y facilitar el mantenimiento:

- **Capa de presentación (UI)**: Gestiona la interacción con el usuario. Puede ser una página web (ASP.NET MVC/Razor), una app de escritorio (WPF, MAUI) o una app móvil.
- **Capa de negocio (Business Layer)**: Contiene la lógica de negocio, reglas, validaciones, cálculos y procesos clave del sistema. Se implementa mediante clases y servicios.
- **Capa de acceso a datos (Data Access Layer)**: Se encarga de comunicar la aplicación con las bases de datos. Usa tecnologías como Entity Framework o ADO.NET.
- **Capa de servicios / APIs**: Permite exponer funcionalidades a otros sistemas mediante servicios web (REST, gRPC, SOAP). ASP.NET Core facilita la creación de APIs robustas.
- **Modelo MVC (Model-View-Controller)**: Patrón muy común en aplicaciones web con ASP.NET. Separa los datos (Modelo), la interfaz de usuario (Vista) y la lógica de control (Controlador).
- **Modelo MVVM (Model-View-ViewModel)**: Usado en aplicaciones WPF, MAUI y Blazor. Separa claramente la lógica de presentación de la vista, mejorando la mantenibilidad.

4.3. PERSISTENCIA DE DATOS EN .NET: ADO.NET Y ENTITY FRAMEWORK

En el ecosistema .NET, el acceso a datos puede realizarse a través de dos enfoques principales:

4.3.1. ADO.NET



ADO.NET es el conjunto de clases que proporciona .NET para acceder y manipular datos desde diversas fuentes (bases de datos relacionales, archivos XML, etc.). Permite un acceso de bajo nivel y control total sobre las operaciones con bases de datos.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Componentes principales:

Clase	Función
SqlConnection	Establece una conexión con una base de datos SQL Server.
SqlCommand	Ejecuta comandos SQL o procedimientos almacenados.
SqlDataReader	Recupera datos de forma rápida y hacia adelante (streaming).
DataSet	Contenedor en memoria de múltiples DataTable, puede funcionar desconectado.
DataAdapter	Puente entre la base de datos y el DataSet. Permite sincronizar cambios.

4.3.2. Entity Framework (EF)



Entity Framework es el ORM (Object-Relational Mapper) oficial de .NET. Permite trabajar con **objetos de dominio** sin necesidad de escribir SQL manualmente para la mayoría de operaciones.

- Reduce el código necesario para acceder a los datos.
- Gestiona automáticamente la conexión y la conversión entre objetos y tablas.
- Permite definir relaciones, restricciones y reglas de negocio mediante clases y atributos.
- Facilita las operaciones CRUD con pocas líneas de código.
- Soporta **LINQ (Language Integrated Query)** para realizar consultas con sintaxis C#.

En el desarrollo con .NET, especialmente en arquitecturas empresariales limpias y desacopladas, se utilizan con frecuencia objetos simples conocidos como **POCOs (Plain Old CLR Objects)**. Estos objetos permiten representar datos o entidades sin introducir dependencias innecesarias con los marcos de trabajo utilizados.

POCO (Plain Old CLR Object) es un objeto de C# simple, que no hereda de ninguna clase específica del framework .NET ni implementa comportamientos complejos por defecto. El término deriva de **POJO (Plain Old Java Object)**, utilizado en el ecosistema Java.

Propiedades del objetoPOCO:

- Contiene únicamente **propiedades públicas**, con get y set.
- Puede incluir **métodos, eventos y delegados**, si son necesarios para representar comportamientos básicos.
- Puede **heredar de otras clases** o **implementar interfaces**, aunque no está obligado a hacerlo.
- **No contiene anotaciones, atributos ni dependencias** de frameworks como Entity Framework, ASP.NET, etc.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

En **Entity Framework**, los objeto POCO se utilizan para representar **entidades** que mapean a tablas de la base de datos. El framework detecta estas clases automáticamente (por convención o configuración) y genera las estructuras necesarias para consultar, insertar o actualizar datos.

4.4. SEGURIDAD EN .NET

ASP.NET Identity

- Sistema de gestión de usuarios y roles para aplicaciones ASP.NET y ASP.NET Core.
- Permite registrar, autenticar y gestionar usuarios con soporte para contraseñas seguras, validación de email, 2FA, etc.
- Se puede almacenar la información en bases de datos SQL y personalizar completamente el modelo de usuarios.

4.5. ANÁLISIS DEL MODELO DE ACCESO A DATOS ADO.NET

4.5.1. Biblioteca de Clases Base (BCL)

La **Biblioteca de Clases Base** (**Base Class Library o BCL**) es un conjunto extenso de **clases, interfaces y tipos** que proporcionan **funcionalidades básicas y comunes** que cualquier aplicación puede necesitar.

La BCL actúa como la **caja de herramientas estándar** para los desarrolladores .NET, agrupando miles de clases reutilizables organizadas por espacios de nombres **(System.*)**.

Funcionalidades

Área funcional	Ejemplos de clases o espacios de nombres
Tipos primitivos y colecciones	System.Int32, System.String, System.Collections.Generic.List<T>
Entrada/salida de archivos	System.IO.File, System.IO.Stream
Acceso a red	System.Net.Http, System.Net.Sockets
Seguridad y cifrado	System.Security.Cryptography
Acceso a bases de datos (ADO.NET)	System.Data.SqlClient, System.Data.DataSet
Manipulación de XML	System.Xml, System.Xml.Linq
Multitarea y concurrencia	System.Threading, System.Threading.Tasks
Serialización	System.Text.Json, System.Runtime.Serialization

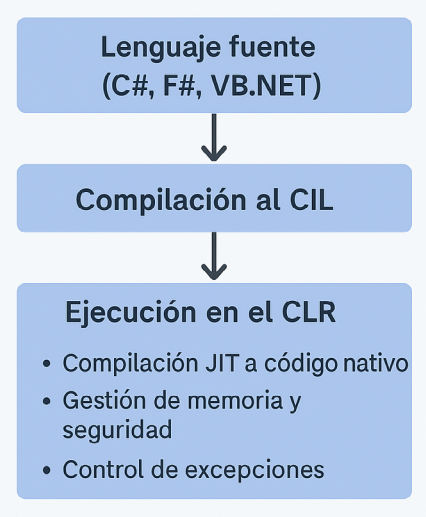
4.5.2. Common Language Runtime (CLR)



El **Common Language Runtime (CLR)** es el **motor de ejecución** del framework .NET. Es el responsable de **ejecutar el código compilado a CIL (Common Intermediate Language)** y ofrecer un entorno seguro, gestionado y eficiente para las aplicaciones.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Todos los lenguajes .NET (C#, VB.NET, F#, etc.) compilan su código fuente a **CIL (Common Intermediate Language)**. El CLR **ejecuta ese código**, independientemente del lenguaje en que fue escrito, garantizando:



- **Ensamblado (Assembly):** unidad básica de despliegue, versionado y seguridad en .NET (archivos **.dll** o **.exe**). Contienen un **Manifiesto** con los metadatos.
- **GAC (Global Assembly Cache):** caché de máquina donde se almacenan los **ensamblados compartidos** para que puedan ser usados por varias aplicaciones simultáneamente.

4.5.3. Lenguajes y compilación al formato intermedio común (CIL)

El **framework .NET** está diseñado como una **plataforma multilenguaje**, lo que significa que permite desarrollar aplicaciones utilizando distintos lenguajes de programación —como **C#**, **Visual Basic .NET (VB.NET)** o **F#**—, siempre que se ajusten a un conjunto común de normas y estructuras definido por el **Common Language Specification (CLS)**.

- **Proceso de compilación a CIL:** Todos estos lenguajes son compilados, no directamente a código máquina, sino a un formato intermedio común llamado CIL
  - **CIL (Common Intermediate Language)**, también conocido anteriormente como **MSIL (Microsoft Intermediate Language)**. Este CIL es independiente del sistema operativo o la arquitectura del procesador, y actúa como un paso intermedio antes de la ejecución real del programa.
- **Ejecución en el CLR. Cuando se ejecuta una aplicación, el Common Language Runtime (CLR) se encarga de:**
  1. **Compilar Just-In-Time (JIT)** el código CIL a **código máquina nativo**, optimizado para el entorno donde se ejecuta.
  2. Gestionar la **memoria**, el **recolector de basura (GC)**, los **hilos**, la **seguridad** y el **manejo de excepciones**.
  3. Garantizar que el código se ejecute de forma **segura, eficiente y aislada** del hardware subyacente.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

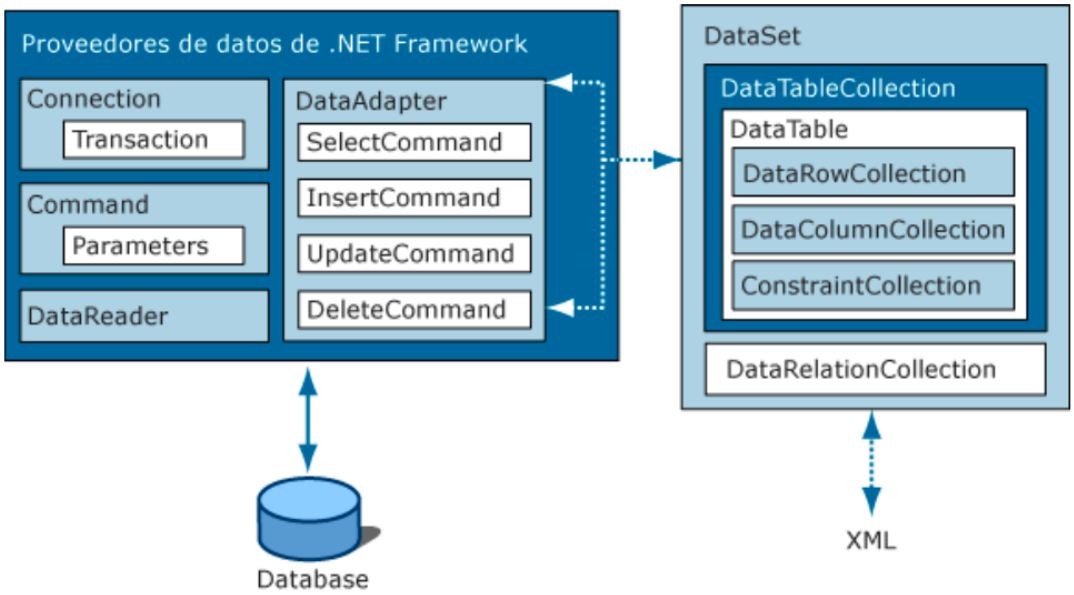
Esto permite que el desarrollo con .NET sea **portátil entre plataformas** con gran **interoperabilidad entre lenguajes** y con capacidades avanzadas de **rendimiento y diagnóstico**.

Lenguajes de programación

Lenguaje	Tipo	Uso principal
C#	Lenguaje orientado a objetos	Desarrollo general, backend, escritorio, web
VB.NET	Lenguaje orientado a objetos	Aplicaciones de escritorio, formularios
F#	Lenguaje multiparadigma	Para ciencia de datos, procesamiento paralelo, programación declarativa o desarrollo de software complejo con alta expresividad
C++/CLI	Extensión de C++ para CLR	Interoperabilidad con librerías nativas
JScript .NET	Derivado de JavaScript	Aplicaciones web (obsoleto)
IronPython	Implementación de Python	Automatización, scripting
IronRuby	Implementación de Ruby	Desarrollo ágil, scripting
PowerShell	Shell de automatización	Administración de sistemas, scripts
Boo	Estilo Python con sintaxis .NET	Scripting, prototipado rápido
COBOL.NET	Versión de COBOL para CLR	Modernización de aplicaciones legacy
Delphi Prism (Oxygene)	Lenguaje Pascal para .NET	Aplicaciones .NET con estilo Pascal

4.5.4. ADO.NET

ADO.NET es el modelo de acceso a datos del .NET Framework. Permite conectar una aplicación con una fuente de datos (una base de datos relacional como SQL Server) y realizar operaciones como consultar, insertar, actualizar o eliminar información.



4.5.4.1. Proveedores de datos (lado izquierdo del diagrama)

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Los **proveedores de datos ADO.NET** son un conjunto de clases que interactúan directamente con la base de datos. Incluyen los siguientes elementos:

- **Connection:** Abre y mantiene la conexión con la base de datos. **Ejemplo: SqlConnection.**
    - **Transaction:** Gestiona transacciones sobre una conexión activa para garantizar atomicidad.



- **Command:** Representa una instrucción SQL o procedimiento almacenado que se ejecuta sobre la base de datos. **Ejemplo: SqlCommand.**
    - **Parameters:** Permite definir parámetros para consultas parametrizadas, evitando SQL Injection.

- **DataReader:** Permite recorrer los resultados de una consulta de forma rápida y hacia adelante (solo lectura). **Ejemplo: SqlDataReader.**
- **DataAdapter:** Puente entre la base de datos y el DataSet.
    - **Contiene 4 comandos:**
        - SelectCommand: recupera datos.
        - InsertCommand: inserta nuevos datos.
        - UpdateCommand: modifica datos existentes.
        - DeleteCommand: elimina datos.

Un **proveedor de datos ADO.NET** es un conjunto de clases especializadas para acceder a una fuente de datos específica.

Proveedores comunes

Proveedor	Fuente de datos
System.Data.SqlClient	SQL Server
System.Data.OleDb	OLE DB
System.Data.Odbc	ODBC
MySql.Data.MySqlClient	MySQL (externo)
Npgsql	PostgreSQL

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

Clases principales del modelo conectado

Clase	Función
SqlConnection	Establece la conexión con la base de datos.
SqlCommand	Ejecuta instrucciones SQL o procedimientos almacenados.
SqlDataReader	Lee datos de forma rápida y hacia adelante.
SqlTransaction	Gestiona transacciones explícitas.
SqlParameter	Permite consultas parametrizadas seguras.

4.5.4.2. DataSet y modelo desconectado (lado derecho del diagrama)

El **DataSet** representa un contenedor en memoria que puede almacenar múltiples tablas, relaciones y restricciones. No requiere conexión activa con la base de datos una vez que se ha cargado la información.

- **DataTableCollection.** Contenedor de varias tablas (DataTable) dentro del DataSet.
- **DataTable.** Representa una tabla de datos con filas (DataRow) y columnas (DataColumn). Dentro del DataTable hay:

Componente	Función
DataRowCollection	Conjunto de filas (registros)
DataColumnCollection	Conjunto de columnas (campos)
ConstraintCollection	Reglas de integridad (por ejemplo, claves primarias)

- **DataRelationCollection:** Define las relaciones entre tablas del DataSet, similar a claves foráneas en bases de datos relacionales.

Objeto	Descripción
DataSet	Contenedor de datos en memoria.
DataTable	Representa una tabla con filas (DataRow) y columnas (DataColumn).
DataRelation	Define relaciones entre tablas dentro del DataSet.
Constraint	Define reglas de integridad (clave primaria, foránea, etc.).

ADO.NET permite leer y escribir datos en formato **XML** mediante métodos como **DataSet. ReadXml() y WriteXml().**

4.6. GESTIÓN DE PAQUETES Y TIPOS DE DATOS EN .NET

El desarrollo moderno en .NET requiere dominar aspectos fundamentales como la gestión de dependencias externas mediante **NuGet** y el uso adecuado de **tipos de datos opcionales (nullable)** en C#, especialmente útil en contextos de desarrollo backend.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025



NuGet es el **gestor de paquetes oficial para .NET**. Permite añadir bibliotecas de terceros a un proyecto mediante comandos de consola, lo que simplifica enormemente la gestión de dependencias, actualizaciones y compatibilidad.

Instalación desde CLI (interfaz de línea de comandos). Para instalar un paquete NuGet en un proyecto .NET se utiliza el siguiente comando:

```
dotnet add package <NombreDelPaquete>
```

Ejemplo. Para instalar la biblioteca iText (utilizada para generar documentos PDF), se usaría:

```
dotnet add package iText
```

Esto añade la referencia en el archivo .csproj y descarga el paquete desde nuget.org.

5. LENGUAJES, TECNOLOGÍAS Y HERRAMIENTAS

El desarrollo de aplicaciones empresariales en plataformas como Jakarta EE y .NET implica el uso de múltiples **lenguajes de programación**, **tecnologías auxiliares** para el intercambio de datos y servicios, así como **herramientas de desarrollo** que facilitan el diseño, implementación, prueba y despliegue de los sistemas.

5.1 LENGUAJES

Java

- **Lenguaje principal en Jakarta EE**.
- **Orientado a objetos**, robusto, con una sintaxis clara y fuerte comunidad.
- Permite desarrollar **aplicaciones multiplataforma** gracias a la JVM.
- Usado en todos los niveles: desde Servlets hasta entidades JPA y servicios REST.

C#

- **Lenguaje principal de la plataforma .NET.**

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

- Moderno, seguro y expresivo. Soporta programación orientada a objetos, funcional, asincrónica y basada en eventos.
- Evoluciona rápidamente con nuevas versiones de .NET, incorporando características modernas como expresiones lambda, LINQ y pattern matching.
- En .NET, un **namespace** (espacio de nombres) es una **forma de organizar y agrupar clases, interfaces, estructuras, enumeraciones y otros tipos bajo un identificador lógico**. Facilita la lectura, mantenimiento y reutilización del código, además de evitar conflictos entre nombres iguales en distintos contextos. Aunque los namespaces son una característica del framework .NET, su uso y declaración **se hace directamente en el lenguaje C#**

VB.NET (Visual Basic .NET)

- **Lenguaje orientado a objetos que también forma parte de .NET.**
- Su sintaxis más simple lo hace accesible, aunque su uso ha disminuido frente a C#.
- Es compatible con el CLR y puede usarse para desarrollar aplicaciones empresariales, aunque ya no recibe mejoras importantes.

5.2 HERRAMIENTAS DE DESARROLLO

Eclipse (Java)

- Entorno de desarrollo integrado (IDE) gratuito y de **código abierto.**
- Ampliamente **usado en el desarrollo con Jakarta EE.**
- Soporta plugins para Maven, Git, servidores de aplicaciones, testing, etc.

IntelliJ IDEA (Java)

- IDE profesional desarrollado por JetBrains.
- Muy valorado por su productividad, asistencia inteligente al código y depuración.
- Disponible en versión Community (gratuita) y Ultimate (de pago, con soporte completo a Jakarta EE, Spring, etc.).



Visual Studio (C#, VB.NET)

- **IDE oficial de Microsoft para el desarrollo .NET.**
- Potente y completo, con herramientas de diseño visual, debugging, pruebas, gestión de dependencias (NuGet), integración con Azure, etc.
- Su versión **Visual Studio Code**, más ligera y multiplataforma, es muy usada también para proyectos .NET Core y front-end.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

6. DESARROLLO DE INTERFACES DE USUARIO

La **interfaz de usuario (UI)** es la parte de la aplicación que interactúa directamente con el usuario final. Su diseño debe ser funcional, intuitivo y accesible.

6.1 JAVA EE/JAKARTA EE

JSP (JavaServer Pages)

- Tecnología para generar contenido HTML dinámico desde el servidor.
- Usa etiquetas especiales **(<% ... %>)** para insertar lógica Java en las páginas.
- Se traduce internamente a Servlets.
- Hoy en día **se considera obsoleta** frente a alternativas más modernas como JSF o frameworks JavaScript.

JSF (JavaServer Faces)

- **Framework estándar de Java EE para construir interfaces web** basadas en componentes reutilizables.
- Usa archivos .**xhtml** para definir la vista y beans de respaldo (Managed Beans) para la lógica.
- Ofrece validación, navegación, gestión del estado y enlaces a modelos de datos.

PrimeFaces

- Biblioteca de componentes UI para JSF muy popular.
- Proporciona más de 100 componentes visuales (gráficas, menús, tablas, etc.) con aspecto moderno y soporte AJAX.
- Simplifica la creación de interfaces ricas sin necesidad de JavaScript.

Jakarta EE puede integrarse con otras soluciones como Vaadin (para UIs tipo SPA) o Thymeleaf (plantillas más modernas), pero JSF sigue siendo la solución oficial.

6.2 .NET

Las **plantillas empresariales** son una herramienta utilizada en entornos de desarrollo como **Visual Studio .NET** para definir y aplicar una **política de arquitectura común** en el desarrollo de aplicaciones.

- Su uso impone una **disciplina de diseño y estructura** al equipo de desarrollo.
- Permiten iniciar proyectos con una base predefinida que incluye capas, dependencias y convenciones.
- Favorecen la **uniformidad, mantenibilidad y reutilización** del código en equipos grandes o proyectos complejos.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025

ASP.NET WebForms

- Tecnología **antigua de Microsoft** para construir interfaces web con separación entre código y presentación.
- Aunque funcional, **ha sido sustituida** progresivamente por tecnologías más modernas.

ASP.NET MVC

- Introduce el **patrón arquitectónico Modelo-Vista-Controlador**.
- Promueve la separación de responsabilidades: lógica de presentación, de negocio y de acceso a datos.
- Mejora la capacidad de realizar **pruebas unitarias** y facilita el mantenimiento.
- Más cercano al control del flujo y estructura del código por parte del desarrollador.

ASP.NET Web Pages

- Modelo más reciente y **simplificado**, pensado para desarrolladores web con poca experiencia en ASP.NET.
- Muy fácil de aprender, permite mezclar HTML y C# en archivos .cshtml.
- Ideal para **prototipos rápidos** o aplicaciones ligeras sin mucha complejidad arquitectónica.



Razor

- **Razor** es una **sintaxis de plantillas de ASP.NET Core** que permite combinar **HTML con código C#** para generar contenido dinámico del lado del servidor. Se usa típicamente en **vistas (.cshtml)** de aplicaciones web MVC o Razor Pages.
- Usada en **ASP.NET MVC** y **Razor Pages**, mejora la claridad y separación de lógica y diseño.

Blazor

- Framework innovador para crear aplicaciones web interactivas usando solo C# y .NET, sin JavaScript.
- Puede ejecutarse en el **servidor** (Blazor Server) o directamente en el navegador con **WebAssembly** (Blazor WebAssembly).
- Permite reutilizar lógica de negocio entre cliente y servidor.

WPF (Windows Presentation Foundation)

- Tecnología de escritorio para Windows con soporte para interfaces gráficas avanzadas, animaciones y multimedia.
- Utiliza **XAML** para definir la interfaz y el patrón **MVVM** para separar lógica y vista.

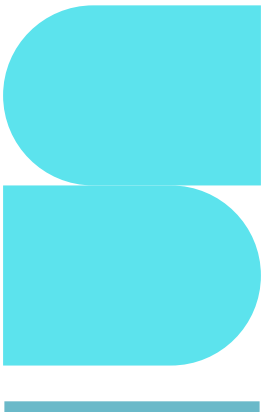
© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025



MAUI (.NET Multi-platform App UI)

- **.NET MAUI** es un **framework de desarrollo multiplataforma** que permite crear **aplicaciones móviles (Android, iOS)** y **de escritorio (Windows, macOS)**. Es la evolución de Xamarin.Forms dentro del ecosistema .NET.
- Usa XAML y C#, y es ideal para aplicaciones móviles multiplataforma.
- **Framework unificado para crear interfaces nativas** en **Windows, Android, iOS y macOS** desde un único código base.

© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025



© Reservados todos los derechos de esta edición para POWEREDUCATION S.L., 2025.

Queda rigurosamente prohibida cualquier forma de reproducción, distribución comunicación pública o transformación total o parcial de esta obra sin el permiso escrito de los titulares de los derechos de explotación.